

Quality Assurance Engineer

이시영 (Lee Siyoung)

</> Core Competencies

Python

Selenium

Pytest

Test Automation

Jenkins & Git & svn

C++

Jira

Key Projects

AI HelpyChat 자동화 프로그램

Test Automation Framework & QA Strategy

엔픽셀 - 그랑사가 (Gran Saga)

클라이언트 게임 개발 & QA 협업

AI HelpyChat 자동화 프로그램

Test Automation Framework & QA Strategy

프로젝트 개요

본 프로젝트는 **AI Chat 웹 서비스**의 품질을 보장하기 위해 시작되었습니다. 기능, UI, 보안 등 다양한 품질 요소를 체계적으로 검토하고 결함을 관리하는 것이 주 목적입니다.

반복적인 테스트 업무를 효율화하기 위해 **Python과 Selenium**을 기반으로 한 테스트 자동화 프레임워크를 구축하였으며, Pytest를 활용하여 테스트 코드 작성 및 커버리지 확대를 달성했습니다.

특히 **High Priority** 기능인 로그인, 회원가입, 메인 챗봇 화면에 대한 집중적인 검증 시나리오를 설계하여 배포 전 리스크를 최소화하였습니다.

핵심 목표 (Key Objectives)

85% ↑

Pass Rate

95% ↑

실행율 (Execution)

75% ↑

자동화 전환율

90% ↑

High Priority Pass

기술 스택 (Tech Stack)

 Python  Selenium  Pytest  POM Pattern  Jira

맡은 역할 & 상세 책임

메인 챗봇 화면을 중심으로 핵심 기능의 품질을 보증하고,
Python/Selenium 기반의 자동화 프레임워크를 설계 및 구현하여 테스트 효율성을 극대화했습니다.
핵심 책임 : 메인 챗, 메인 화면 담당 + 전반적인 자동화 프로그램 기능 설계 및 구현



담당 모듈

메인 화면 (Main Dashbord)

사용자 진입 시점의 UI 정합성 및 레이아웃 검증

메인 챗봇 (AI Chatbot)

대화 입력, 응답 출력, 컨텍스트 유지 등 핵심 기능

메인 채팅 기능 검증

파일 업로드(지원/미지원 확장자, 단일/다중), 대화 복사/붙여넣기 및 전송



주요 책임

테스트 설계

테스트 시나리오 및 케이스(TC) 상세 작성

자동화 프레임워크 구축

Selenium & Pytest 활용, POM 패턴 설계 및 구현

테스트 코드 작성

메인 화면/메인 챗봇 테스트 코드 전담 작성
Manager 파일(파일 관리) 구현
Utils(공용 모듈, WebDriver 설치/옵션) 구현
DDT 구현(JSON 기반, 데이터와 로직 분리)
로깅 시스템 구현(CSV 파일 저장)



협업 및 산출물

Bug Reporting

Jira를 이용한 상세 버그 리포트 및 생명주기 관리

Test Reporting

테스트 실행 결과 분석 및 최종 결과 보고서 작성

협업 커뮤니케이션

팀원들과 정기적 스크럼/스프린트를 통해 이슈 공유

테스트 계획서 작성 (Test Plan)

체계적인 품질 검증을 위해 테스트 유형별 전략을 수립하고,
프로젝트 종료를 위한 정량적 완료 조건 및 결함 관리 기준을 명확히 정의합니다.

테스트 전략 (Strategy)

테스트 범위 및 유형

- **기능 테스트:** 주요 프로세스 및 비즈니스 로직 정상 동작 확인
- **UI/UX 테스트:** 화면 레이아웃, 반응형 디자인 및 컴포넌트 동작 검증

데이터 준비 (Test Data)

- **테스트 계정:** 권한별 계정 생성 (Admin, User, Guest) 및 관리
- **Mock Data:** 시나리오 검증을 위한 사전 조건 데이터 및 Mocking 셋업

완료 조건 & 결함 관리

완료 조건 (Exit Criteria)

85%
Pass Rate 목표

95%
테스트 실행율

90%
High Priority Pass

75%
자동화 전환율

결함 등급 (Defect Severity)

Critical 서비스 불가, 데이터 손실 등 치명적 결함

Major 주요 기능 동작 불가, 우회 불가능

Minor 기능상 문제없으나 UI/UX 불편함 초래

Trivial 단순 오타, 경미한 UI 깨짐

테스트 시나리오 구조

PHC-TS Framework Structure & Scenarios

ID Naming Convention



PHC Project Helpy Chat (프로젝트명)

TS Test Scenario (테스트 시나리오)

00 Sequential Number (일련번호)

본 프로젝트에서는 **총 8개의 핵심 시나리오**를 도출하여 기능별, 보안별로 체계적으로 분류하였습니다.

ID 규칙은 프로젝트명 약어와 시나리오 식별자를 조합하여 **PHC-TS-00** 형식으로 표준화하였으며, 이를 통해 이슈 추적과 문서 관리의 효율성을 높였습니다.

☰ 주요 시나리오 목록

PHC-TS01 로그인

PHC-TS02 회원가입

PHC-TS03 AI 에이전트

PHC-TS04 도구 메뉴

PHC-TS05 메인 챗봇

PHC-TS06 프로필 관리

PHC-TS07 간단 보안테스트

🎯 테스트 목적 (Objectives)



핵심 경로 성공 보장

사용자가 가장 빈번하게 사용하는 주요 기능(로그인, 대화 등)이 기획 의도대로 정상 동작하는지 확인하여 기본 사용성을 보장합니다.



예외 및 보안 경계 (Boundary) 검증

잘못된 입력값 처리, 비정상적인 접근, 악의적인 스크립트 주입 등 잠재적인 위협과 예외 상황에 대한 방어 로직을 검증합니다.

테스트 시나리오 소개

AI-Helpy-Chat Test Scenario Overview & Strategy

🎯 전략 및 완료 기준 (Strategy & Exit Criteria)

✔ 테스트 범위

로그인/회원가입, 메인 챗봇, AI 에이전트, 결제 등 주요 기능

📊 리스크 산정 (Risk Assessment)

High(핵심 기능), Medium(주요 기능), Low(부가 기능) 분류

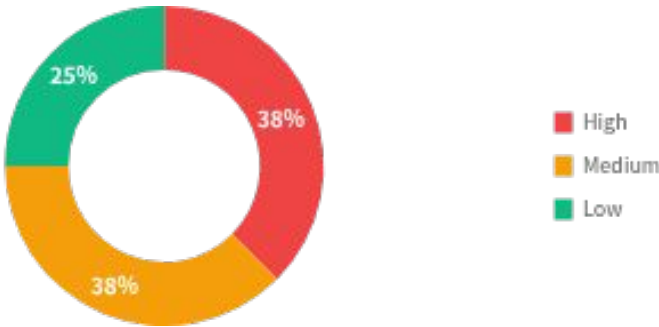
Scenario Coverage

100% 완료

Critical Path

Pass 필수

📊 시나리오 우선순위 분포



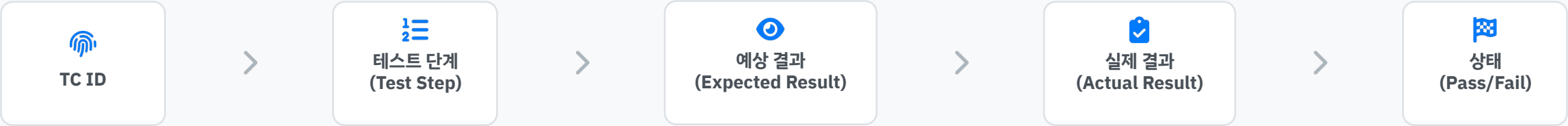
📋 AI-Helpy-Chat 테스트 시나리오 (Test Scenarios)

ID	시나리오명	목적	테스트 범위 (URL)	유형	우선순위	담당자
PHC-TS01	사용자 로그인/로그아웃	사용자가 로그인하여 서비스를 이용하고 로그아웃할 수 있는지 검증한다.	/accounts/signin	자동	High	이성진
PHC-TS02	사용자 회원 가입	사용자가 헬피챗 플랫폼에 회원 가입을 할 수 있는지 검증한다.	/accounts/signup	자동	High	이수진
PHC-TS03	메인 화면	ai-helpy-chat 메인 화면에서 사용자가 사용할 수 있는 기능과 성능을 검증한다.	/ai-helpy-chat	자동수동	High	이시영
PHC-TS04	도구 메뉴	AI 사용을 더 간편하게 해주는 도구를 사용자가 설정할 수 있는지 검증한다.	/ai-helpy-chat/tools	자동수동	Medium	이성진
PHC-TS05	사용자 계정 관리	사용자가 사용하는 계정을 수정 / 관리할 수 있는지 검증한다.	/members	자동수동	Low	정미슬
PHC-TS06	AI 에이전트 메뉴	AI 에이전트와 관련된 기능을 사용자가 이용하고 관리할 수 있는지 검증한다.	/ai-helpy-chat/agents	자동수동	Medium	이수진
PHC-TS07	보안 테스트	SQL Injection, XSS 등을 활용해 간단한 보안 점검을 수행한다.	로그인, AI 대화창, 입력창 등	자동	Medium	정미슬
PHC-TS08	크레딧 구매/사용	사용자가 크레딧을 구매하고 사용할 수 있는지 검증한다.	/payments/credit	제외	Low	이성진

테스트 케이스 구조 & 예시

Standardized Test Case Structure & Execution Examples

테스트 케이스 표준 구조 (Standard Structure)



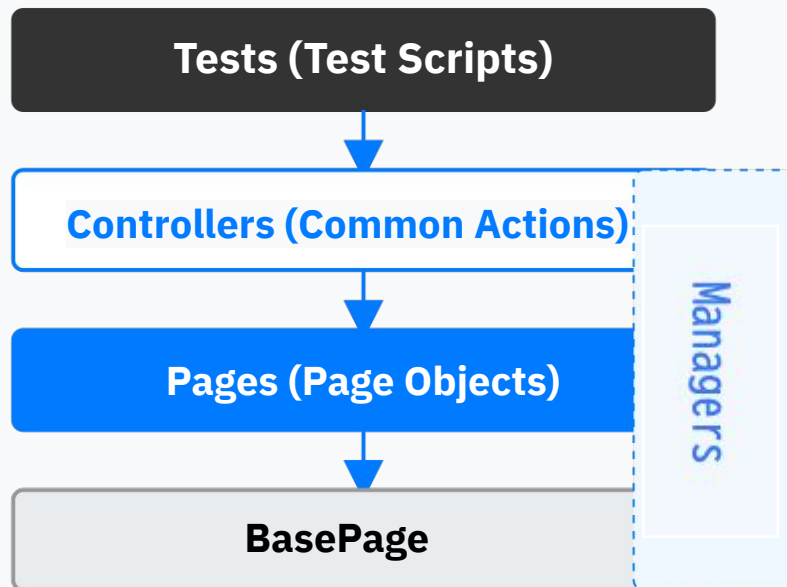
실제 테스트 케이스 실행 예시 (Execution Examples)

60	홈 화면		정상	PHC-TS03-TC030	지원되는 형식의 파일을 정상적으로 업로드	자동	AI가 업로드한 파일을 정확하게 파악하여 관련된 답변을 제공하는지 확인	사용자가 로그인된 상태
61				PHC-TS03-TC031	jpg/png 형식의 이미지를 정상적으로 업로드	자동	AI가 업로드한 이미지를 정확하게 파악하여 관련된 답변을 제공하는지 확인	사용자가 로그인된 상태

1. Helpy Pro Agent 모델 선택 2. '+' 버튼 클릭 3. '파일업로드' 버튼 클릭 4. 지원하는 형식의 파일 선택 5. '업로드' 버튼 클릭 6. enter 또는 '보내기' 버튼 클릭	-	AI가 업로드된 파일 정보를 판별하여 해당 파일과 관련된 정보를 제공함	사용자가 보낸 파일과 관련된 응답을 제공한다.	PASS	High	이시영
1. Helpy Pro Agent 모델 선택 2. '+' 버튼 클릭 3. '파일업로드' 버튼 클릭 4. 이미지 파일 선택 5. '업로드' 버튼 클릭 6. enter 또는 '보내기' 버튼 클릭	-	AI가 업로드된 이미지 정보를 판별하여 이미지와 관련된 정보를 제공함	사용자가 보낸 이미지와 관련된 응답을 제공한다.	PASS	High	이시영

POM

구조 설계



Pages 계층 (UI 요소)

화면별 UI 요소(Locator)와 상호작용 메서드를 캡슐화하여 테스트 코드와 UI 정의를 분리했습니다. 이를 통해 UI 변경 시 유지보수 비용을 최소화하고 재사용성을 높였습니다.

Controllers 계층 (공통 행동)

입력, 클릭, 클립보드 복사 등 여러 페이지에서 반복적으로 사용되는 비즈니스 로직을 모듈화했습니다. Static Method로 구현하여 인스턴스 생성 오버헤드를 줄이고 코드 중복을 제거했습니다.

BasePage & Managers

BasePage는 WebDriverWait 및 요소 탐색(presence, visibility, clickable) 공통 로직을 추상화하여 안정성을 확보합니다. Managers는 파일 입출력 및 스크린샷 저장을 전담 관리합니다.

Controllers 구조

효율적인 자동화를 위한 공통 행동 모듈 설계



모듈화 (Modularity)

여러 페이지에서 반복적으로 발생하는 동일한 사용자 행동(입력, 클릭, 복사 등)을 **공통 로직으로 분리**하여 관리합니다. 특정 Page Object에 종속되지 않아 유지보수가 용이하고 재사용성이 뛰어납니다.

```
class ChatInputController:

    @staticmethod
    def send_text(element, text: str):
        element.click()
        element.send_keys(text)

    @staticmethod
    def paste_text(element, text: str):
        ClipboardController.copy(text)
        element.click()
        ClipboardController.paste(element)
```



효율성 (Efficiency)

'행동' 자체는 상태(State)를 가질 필요가 없으므로 **@staticmethod**로 구현했습니다. 이를 통해 불필요한 인스턴스 생성을 방지하고 메모리 오버헤드를 제거하여 실행 속도를 최적화했습니다.

```
class ClipboardController:

    @staticmethod
    def copy(text: str):
        pyperclip.copy(text)

    @staticmethod
    def paste(element):
        system = platform.system()
        if system == "Darwin": # 맥용!
            element.send_keys(Keys.COMMAND, "V")
        else: # 윈도우 용!
            element.send_keys(Keys.CONTROL, "V")
```



사용 예시 (Usage)

테스트 코드 내에서 인스턴스화 없이 간결하게 호출할 수 있습니다. 복잡한 Action Chains 로직도 한 줄로 처리 가능합니다.

```
textarea = WebDriverWait(self.driver, WAIT_TIME).until(
    EC.visibility_of_element_located((By.XPATH, XPATH["BTN_EDIT_AREA"])))

ai_input_last = self.fm.read_json_file("ai_text_data.json")[ChatKey.RENAME][-1]
content = ai_input_last["content"]

ChatInputController.reset_text(textarea)
ClipboardController.copy(content)
ClipboardController.paste(textarea)
```

자동화 안정화 - Explicit Wait

WebDriverWait을 활용한 명시적 대기와 커스텀 핸들러를 도입하여, 네트워크 지연이나 UI 렌더링 속도 차이로 인한 '**Flaky Test**'를 원천 차단하고 스크립트 신뢰성을 확보했습니다.

1. 동적 대기 (Explicit Wait)

요소의 상태(Visibility/Clickable)에 따라 적절한 대기 조건을 동적으로 적용하여 불필요한 대기 시간을 최소화합니다.

2. 기능 추상화 (Abstraction)

자주 사용되는 탐색 및 대기 로직을 유틸리티 함수로 래핑하여, 코드 중복을 제거하고 가독성을 높였습니다.

3. 강제 실행 (JS Executor)

UI 가림 현상 등으로 일반 클릭이 불가능할 경우, JavaScript로 강제 이벤트를 발생시켜 테스트 실패를 방지합니다.

```
elif option == "visibility":
    return wait.until(EC.visibility_of_element_located((by, value)))

elif option == "clickable":
    return wait.until(EC.element_to_be_clickable((by, value)))
except (TimeoutException, NoSuchElementException): ...

def get_element_by_id(self, id, option="presence", timeout = TIMEOUT_MAX):
    return self.get_element(By.ID, id, option, timeout)
```

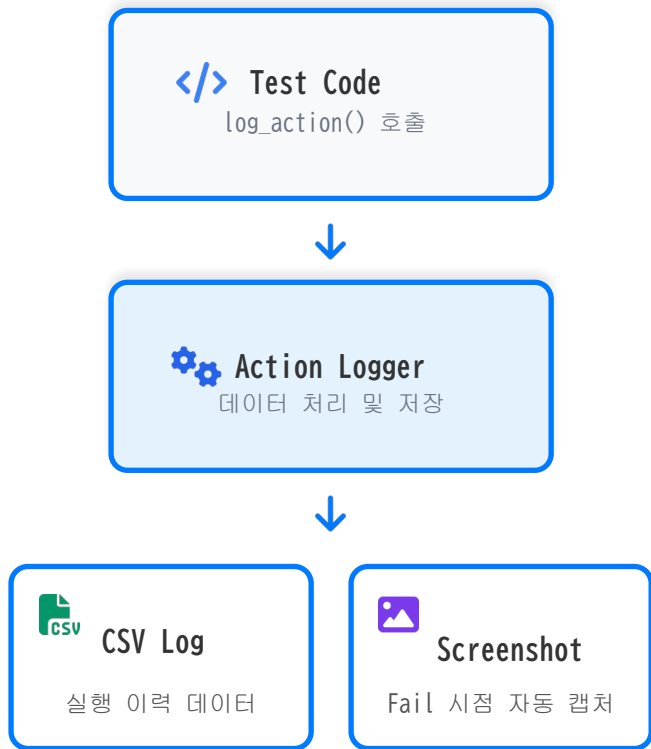
```
# 1. (탐색) 공용함수로 탐색해서 가져옴
first_item = chat_items[ITEM_INDEX] # chat_items = self.get_elements..

# 2. (액션) 이미 찾은 element가 액션 가능한지 대기
WebDriverWait(self.driver, WAIT_TIME).until(EC.element_to_be_clickable(first_item))
self.driver.execute_script("arguments[0].click();", first_item)

# 3. (액션 이후) UI가 사라졌는지 대기
WebDriverWait(self.driver, WAIT_TIME).until(EC.invisibility_of_element_located((By.
```

1. element를 탐색하기 위해 **WebDriverWait** 사용

2. 엘리먼트를 찾는 대기, 액션을 위한 대기, 액션 이후 UI 전환을 기다리는 대기 분리



Action Logger 시스템

로깅 항목 (Data Structure)

테스트 실행의 모든 단계를 추적하기 위해 5가지 핵심 데이터를 기록합니다.

- ✓ **Context:** 화면 및 액션 (예: Main_Home/Click_Login)
- ✓ **Result:** 성공/실패 여부 (Pass / Fail)
- ✓ **Elapsed Time:** 단계별 소요 시간 (성능 지표 활용)
- ✓ **Screenshot Path:** 실패 시 이미지 경로 자동 저장

사용성 및 기대 효과

복잡한 로깅 로직을 log_action 함수 하나로 추상화하여, 테스트 코드의 가독성을 해치지 않으면서 상세한 데이터를 자동으로 축적합니다.

```
log_action(ctx, ActionResult(test_name, TestResult.PASSED, elapsed_time =  
0, detail = "PHC-TS03-TC024"))
```

기대 효과: CSV 필터링을 통해 실패한 케이스만 즉시 추출하여 디버깅 시간을 단축하고, 스크린샷 자동 매핑으로 결함 원인 분석의 리드타임을 획기적으로 줄입니다.

Action Logger- 실행 결과

```
timestamp,test_name,page,action,target,result,detail,elapsed_time,screenshot
2025-12-22 14:02:09,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC017_select_latest_chat,0,
2025-12-22 14:02:09,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,open_selected_edit_menu,0,
2025-12-22 14:02:09,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,click_change_chat_name,0,
2025-12-22 14:02:09,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC020_cancel_edit,0,
2025-12-22 14:02:09,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC017_select_latest_chat,0,
2025-12-22 14:02:10,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,open_selected_edit_menu,0,
2025-12-22 14:02:10,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,click_delete_chat,0,
2025-12-22 14:02:10,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC022 PHC-TS03-TC024 ended,1.515493749999223,
2025-12-22 14:30:23,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,open_selected_edit_menu,0,
2025-12-22 14:30:23,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,click_change_chat_name,0,
2025-12-22 14:30:23,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC020_cancel_edit,0,
2025-12-22 14:30:24,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC017_select_latest_chat,0,
2025-12-22 14:30:24,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,open_selected_edit_menu,0,
2025-12-22 14:30:24,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,click_delete_chat,0,
2025-12-22 14:30:24,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC022 PHC-TS03-TC024 ended,1.5946309589999146,
2025-12-22 14:30:59,test_cancel_past_edit,chat,test_cancel_past_edit,,pass,PHC-TS03-TC017_select_latest_chat,0,
```

test_conversation_scenario_log

timestamp	test_name	page	action	target	result	detail	elapsed_time	screenshot
2025-12-22 14:15:27	test_conversation_scenario	chat	PHC-TS03-TC004		pass	click_btn_home_menu	0	
2025-12-22 14:15:46	test_conversation_scenario	chat	PHC-TS03-TC005		pass	ai_user_chat	0	
2025-12-22 14:15:46	test_conversation_scenario	chat	PHC-TS03-TC024		pass	copy_last_question	0	
2025-12-22 14:15:47	test_conversation_scenario	chat	PHC-TS03-TC024		pass	paste_last_question	0	
2025-12-22 14:15:53	test_conversation_scenario	chat	PHC-TS03-TC012		pass	reset_chat	0	
2025-12-22 14:15:53	test_conversation_scenario	chat	PHC-TS03-TC029		pass	cancel_edit_question	0	
2025-12-22 14:16:04	test_conversation_scenario	chat	PHC-TS03-TC027_PHC-TS03-TC028		pass	edit_question	0	
2025-12-22 14:16:10	test_conversation_scenario	chat	PHC-TS03-TC007		pass	rename_main_chat	0	
2025-12-22 14:16:14	test_conversation_scenario	chat	PHC-TS03-TC006		pass	retry_response	0	
2025-12-22 14:16:14	test_conversation_scenario	chat	PHC-TS03-TC023		pass	copy_last_response	0	
2025-12-22 14:16:14	test_conversation_scenario	chat	PHC-TS03-TC023		pass	paste_last_response	0	
2025-12-22 14:16:14	test_conversation_scenario	chat	PHC-TS03-TC009		pass	delete_main_chat	0	
2025-12-22 14:16:14	test_conversation_scenario	chat	Test_ended		pass	test_conversation_scenario_ended	47.08377649999970	
2025-12-22 14:32:03	test_conversation_scenario	chat	PHC-TS03-TC004		pass	click_btn_home_menu	0	

테스트 결과 요약

자동화 테스트 수행 결과, 주요 기능의 안정성을 검증하고 목표 품질 기준을 달성하였습니다.

총 테스트 케이스

251개

결함 재현율

100%

Pass

223 (88.8%)

Fail

24 (9.6%)

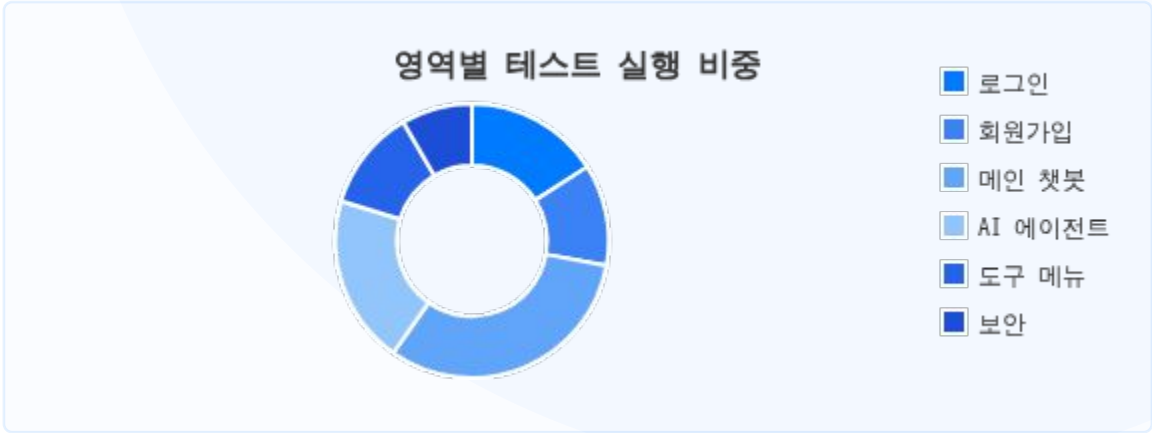
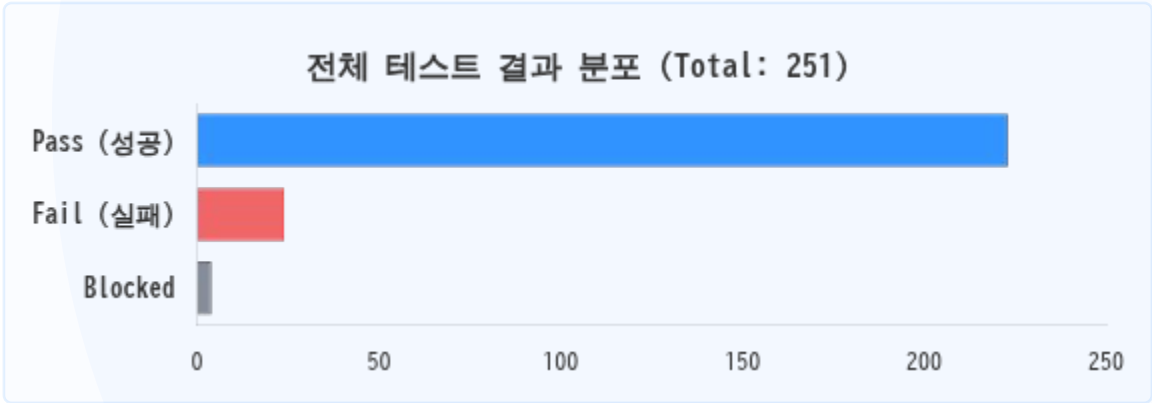
완료 기준 달성 현황

✓ Pass Rate ≥ 88%

✓ 실행율 ≥ 95%

✓ High Priority Pass Rate ≥ 90%

✓ TC 자동화 전환율 ≥ 75%



결함 분석

발견된 결함에 대한 심도 있는 분석 결과입니다. 모든 결함은 재현 및 문서화되었으며, 중요도에 따라 체계적으로 관리되었습니다.

27

Total Defects

발견된 총 결함

100%

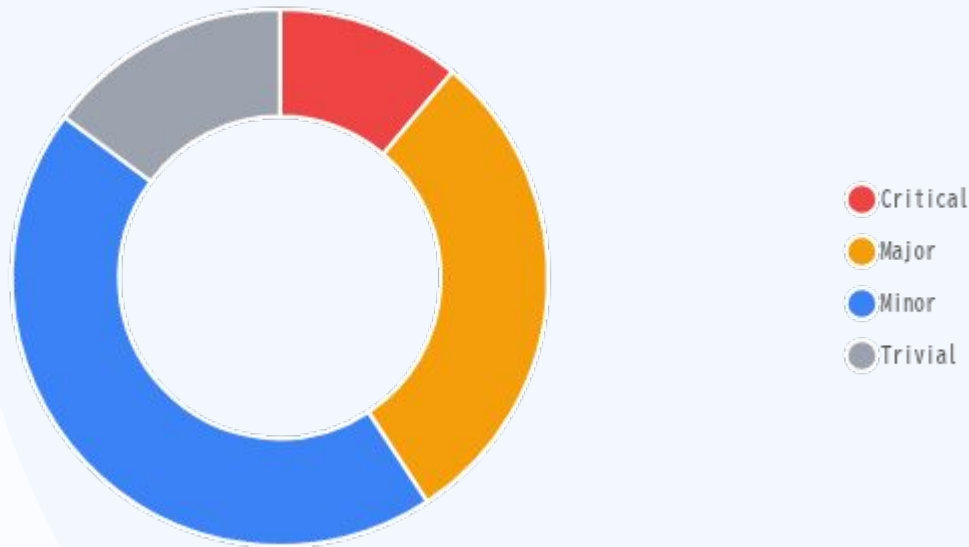
Reproduction Rate

결함 재현율

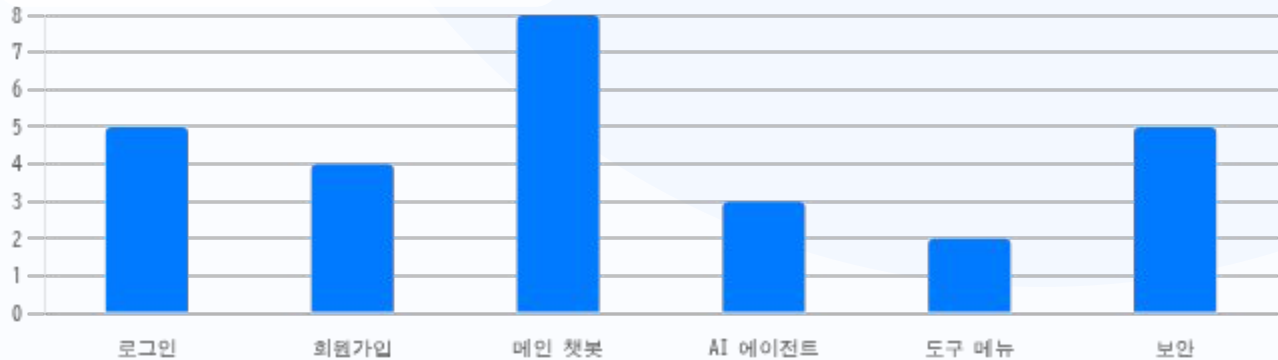
등급별 결함 분포 요약

	Critical (치명적)	3건
	Major (중요)	8건
	Minor (경미)	12건
	Trivial (사소함)	4건

결함 등급 비율 (Severity Distribution)



기능 모듈별 결함 수 (Defects by Module)



[게임 개발] 그랑사가 - 장비 빠른 분해/환원 (게임 개발 & QA 협업)

Request → ACK → Update의 비동기 패킷 흐름을 활용하여 UI 위치에 관계없이 장비 분해/강화/환원 시 인벤토리와 재화 데이터가 실시간으로 동기화되도록 구현했습니다.



White-box 접근 (영향도 분석)



구조 도식화 및 가시화

데이터 변경이 참조되는 UI 컴포넌트의 내부 연결 구조를 도식화하여 QA팀에 제공, 내부 로직의 흐름을 투명하게 공유했습니다.



영향도 분석 (Impact Analysis)

코드가 변경되었을 때 영향을 받는 범위를 기술적으로 특정하여, 불필요한 테스트를 줄이고 핵심 로직에 집중했습니다.



테스트 범위 설정

개발 단계에서 확인된 리스크 영역을 기반으로 테스트 사각지대를 제거하고 정밀한 테스트 범위를 설정했습니다.



Black-box 검증 (사용자 시나리오)



복합 사용자 시나리오 검증

단순 기능 확인을 넘어, 실제 유저가 수행할 법한 복합적인 패턴을 시나리오로 정의하여 검증했습니다.

구매 직후 분해 시도

상점에서 장비를 구매한 직후, 인벤토리 갱신 전 분해 시도 시 동기화 처리 확인

화면 전환 중 데이터 갱신

로딩 화면이나 씬(Scene) 전환 중에 재화 데이터가 변경될 때의 UI 반영 여부

회귀 시나리오 고정

주요 결함 케이스를 체크리스트로 고정하여, 매 빌드 배포 시마다 반복 검증을 수행해 품질 효율을 확보했습니다.

프로젝트 효과



테스트 사각지대 축소



영향 범위 특정



회귀 효율성 증대



[게임 개발] 그랑사가 - 콜라보 펫 시스템 (게임 개발 & QA 협업)

Final Fantasy Collaboration - Pet (Guardian) System

개발 내용 & 범위

- 콜라보 전용 펫
- 이벤트 배너
- 획득 및 성장
- 퀘스트 연동

본 프로젝트는 파이널 판타지 IP와의 콜라보레이션으로, 정해진 이벤트 기간에 맞춰 완벽한 품질을 보장해야 하는 **Hard Deadline** 프로젝트입니다.

새롭게 도입되는 펫(가디언) 시스템의 복잡한 성장 로직과 다양한 퀘스트 연동이 핵심이며, QA 단계 진입 전 개발 단계에서의 선제적인 품질 확보가 필수적이었습니다.



⚙️ 선행 품질 확보 (White-box Strategy)



핵심 로직 단위 테스트 (Unit Test)

이벤트 오픈 리스크 관리를 위해 펫 성장 및 획득 로직에 대한 단위 테스트를 선행 수행하여 로직 결함을 조기에 제거했습니다.



빌드 안정성 확보

QA 진입 전 크리티컬 이슈(Crash, Blocker)를 사전에 차단하여 빌드 안정성을 확보하고 QA 테스트 효율을 높였습니다.

🔍 E2E 테스트 지원 (Black-box Support)



로직 다이어그램 공유 & 기준 제공

펫 획득부터 퀘스트 완료까지 이어지는 전체 로직 흐름을 시각화하여 공유함으로써, QA가 기획 의도를 명확히 검증할 수 있도록 지원했습니다.

퀘스트 완료



배너 진입



펫 획득



성장/강화

[게임 개발] 그랑사가 - 코스튬 상점 리뉴얼 (게임 개발 & QA 협업)

개발 내용: 전체 캐릭터 조회, 옷장 연동, 이벤트 배너, 캐릭터별 분류, 코스튬 소환 및 자동 선택 등 복잡한 상점 로직 개선을 위해 **White-box 로직 공유**와 **Black-box 예외 제안**을 병행했습니다.



로직 공유 (Logic Sharing)

알고리즘 투명화 분류 필터 및 자동 선택(정렬) 내부 처리 로직을 공유받아 블랙박스 테스트의 한계 극복

기대 결과 명확화 복잡한 조합 상황에서도 QA가 명확한 Expected Result를 도출할 수 있도록 기준 수립



예외 케이스 제안

Edge Case 선제 반영 '미보유 캐릭터 코스튬 획득', '기간 만료 배너' 등 명세에 없는 예외 상황 식별

잠재 결함 방지 기획 단계에서 놓칠 수 있는 사용자 시나리오를 QA 관점에서 제안하여 개발 반영



효과 (Effectiveness)

명세 외 결함 예방 개발 로직 이해를 바탕으로 숨겨진 결함을 사전에 차단

검증 효율화 불필요한 커뮤니케이션 비용을 줄이고, 반복 검증 시나리오의 정확도 향상



감사합니다

Thank You for Your Time



자동화 기반
품질 안정화



리스크 우선
검증 전략



데이터 중심
결과 보고

QA Engineer:

이시영 Lee Siyoung

✉ syoung8296@gmail.com